

Transformer (下): Decoder、Cross Attention 与训练技巧

李宏毅《机器学习》2025 第六节：从自回归解码到
exposure bias



基于李宏毅老师课程整理

2026 年 6 月 8 日

视频信息

频道：李宏毅机器学习深度学习课程 (Bilibili)

时长：约 61 分钟

链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=16>

目录

1 Decoder 的任务：把表示变成序列	4
1.1 输出单位：字、subword 或 token	4
1.2 本章小结	4
2 Autoregressive Decoder	4
2.1 Error Propagation	5
2.2 本章小结	5
3 Masked Self-attention 与 END Token	6
3.1 为什么需要 END token	6
3.2 本章小结	7
4 AT 与 NAT：逐步生成还是平行生成	7
4.1 NAT 的输出长度问题	8
4.2 Multi-modality 直觉	8
4.3 本章小结	9
5 Cross Attention: Encoder 与 Decoder 的桥	9
5.1 Q 来自 Decoder, K/V 来自 Encoder	9
5.2 Cross Attention 早于 Transformer	10
5.3 本章小结	11
6 训练 Decoder: Cross Entropy 与 Teacher Forcing	11
6.1 Teacher Forcing	12
6.2 本章小结	13
7 Copy Mechanism 与 Guided Attention	13
7.1 Copy Mechanism	13
7.2 Guided Attention	15
7.3 本章小结	16
8 解码策略: Greedy、Beam Search 与 Sampling	16
8.1 什么时候 Beam Search 有用	17
8.2 Sampling	17
8.3 评价指标与强化学习	18
8.4 本章小结	18

9 Exposure Bias 与 Scheduled Sampling	19
9.1 Scheduled Sampling 的直觉	19
9.2 本章小结	20
10 总结与延伸	21
10.1 拓展阅读	21

1 Decoder 的任务：把表示变成序列

上一讲已经把 Transformer encoder 拆开：encoder 接收输入序列，输出一排上下文化向量。本讲继续讲 decoder。Seq2seq 的完整流程可以写成

$$X \xrightarrow{\text{Encoder}} H \xrightarrow{\text{Decoder}} Y,$$

其中 X 是输入序列， H 是 encoder 输出， $Y = (y_1, \dots, y_N)$ 是 decoder 产生的输出序列。

Decoder 要解决的问题比“分类一次”复杂得多。它不仅要决定每一步输出哪个 token，还要决定什么时候停止。以语音识别为例，输入是一段声音，输出可能是中文字序列“机器学习”。模型不能一开始就知道输出一定是 4 个字；它必须边产生、边根据已经产生的前缀继续预测。

1.1 输出单位：字、subword 或 token

Decoder 的输出单位可以有很多选择。英文常用 subword、wordpiece、BPE token 或 byte-level token；中文课程里为了说明方便，常把一个中文字当作一个 token。实际系统会把所有可能输出组成 vocabulary，大小记作 V 。每一步 decoder 输出一个长度为 V 的分布：

$$p_t = P(y_t \mid y_{<t}, H).$$

这里 p_t 的每个维度对应一个候选 token。模型可以取最大概率 token，也可以从分布中 sampling；训练时则用正确答案监督这个分布。

Decoder 的三个基本问题

Decoder 必须回答三件事：第一，每一步该看哪些过去 token；第二，每一步如何从 encoder 输出抽取相关信息；第三，输出序列何时结束。这三件事分别对应 masked self-attention、cross attention 和 END token。

1.2 本章小结

Decoder 把 encoder 表示转成目标序列。它的输出不是一次性固定维度向量，而是一串 token 分布；每一步都依赖 encoder 信息和已经生成的前缀。

2 Autoregressive Decoder

最常见的 decoder 是 autoregressive decoder，简称 AT decoder。它一次产生一个 token：先读入特殊的 BEGIN token，产生第一个字；再把第一个字当作下一步输入，产生第二个字；如此循环，直到输出 END token。

用概率写就是

$$P(Y \mid X) = \prod_{t=1}^N P(y_t \mid y_1, \dots, y_{t-1}, H).$$

其中 $H = \text{Encoder}(X)$ 。这个分解强调了“前缀条件”：生成第 t 个 token 时，模型只应该依赖已经生成的 $y_1, \dots, y_{t-1}$ ，不能看未来答案。

Autoregressive

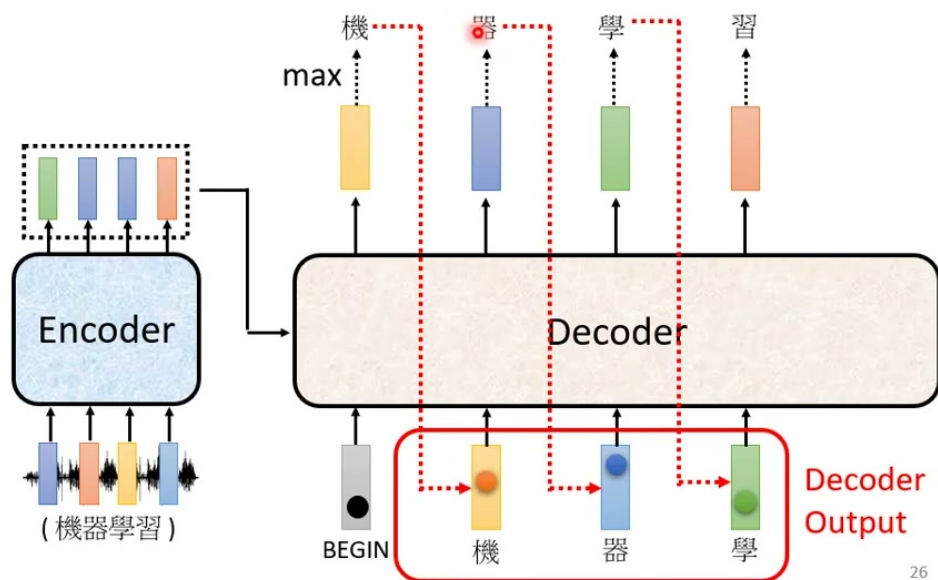


图 1: Autoregressive decoder 会把上一轮输出接回下一轮输入；图中红色虚线表示模型生成“机、器、学”后，再继续产生“习”。²

2.1 Error Propagation

AT decoder 的优势是自然地处理可变长度和前缀依赖；问题是错误会传播。若模型第一步本来应该输出“机”，却输出成“气”，第二步看到的前缀就已经错了。之后模型是在错误上下文上继续预测，可能越走越偏。

这也是后面 exposure bias 和 scheduled sampling 会出现的原因。训练时模型常常看到正确前缀；测试时却只能看到自己的预测前缀。一旦预测错，模型未必知道如何从错误状态恢复。

不要把 AT 解码误解成一次输出整句

推论时，AT decoder 不是同时知道所有输出 token。它先产生第一个，再把第一个放回输入产生第二个。训练图里常画成并行输出，是因为 teacher forcing 可以把正确前缀一次喂进去，但这不是普通推论时的工作方式。

2.2 本章小结

Autoregressive decoder 用链式法则把整句概率拆成逐步生成。它适合可变长度输出，也能利用已经生成的上下文，但会带来错误传播和训练/推论不一致的问题。

²视频画面时间区间：约 08:20–08:50；字幕对应“decoder 看到自己产生出来的错误输入”“error propagation”“一步错步步错”。

3 Masked Self-attention 与 END Token

Transformer decoder 里的第一块 attention 不是普通 self-attention, 而是 masked self-attention。普通 self-attention 允许每个位置看所有位置; decoder 生成第 t 个 token 时, 未来 token 还不存在, 因此位置 t 不能看 $t+1, t+2, \dots$ 。

在实现上, 可以给 attention score 加一个 causal mask:

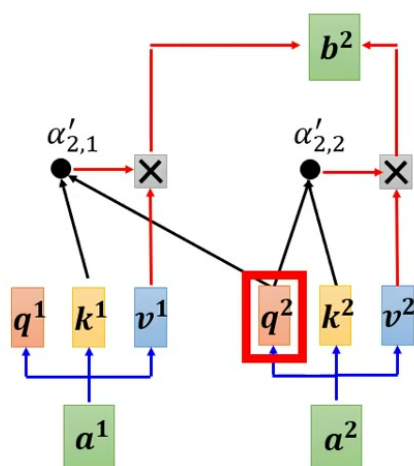
$$e_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}}, \quad \alpha_{ij} = \text{softmax}_j(e_{ij} + M_{ij}),$$

其中

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases}$$

于是第 i 个位置只能 attend 到自己和左边的位置。

Self-attention $\rightarrow$ **Masked Self-attention**



Why masked? Consider how does decoder work

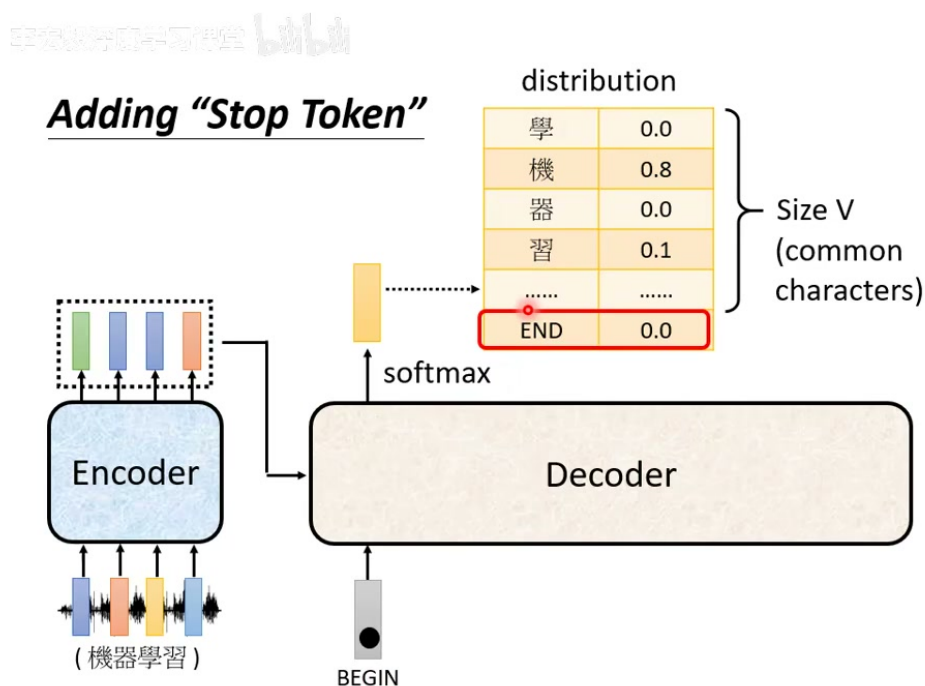
30

图 2: Masked self-attention 中, 计算 b^2 时只能使用 a^1, a^2 , 不能使用还没生成的 a^3, a^4 。³

3.1 为什么需要 END token

Decoder 还要决定什么时候停止。做法是在 vocabulary 里加入一个特殊 token, 例如 END。模型每一步都输出一个分布, END 和普通 token 一样占一个维度。一旦模型输出 END, 生成过程结束。

³视频画面时间区间: 约 12:50-13:20; 字幕对应 “计算 b^2 的时候没有 A^3 跟 A^4 ” “decoder 输出是一个一个产生的, 所以只能考虑左边”。



33

图 3: 除了普通中文字, decoder 的输出分布里还要有 END 这样的特殊符号; 模型输出 END 时, 序列生成停止。⁵

BEGIN 和 END 有时可以使用同一个符号, 因为 BEGIN 只出现在 decoder 输入端, END 只出现在输出端; 也可以使用两个不同符号。关键不是符号名字, 而是训练资料必须告诉模型: 什么时候应该开始, 什么时候应该停止。

3.2 本章小结

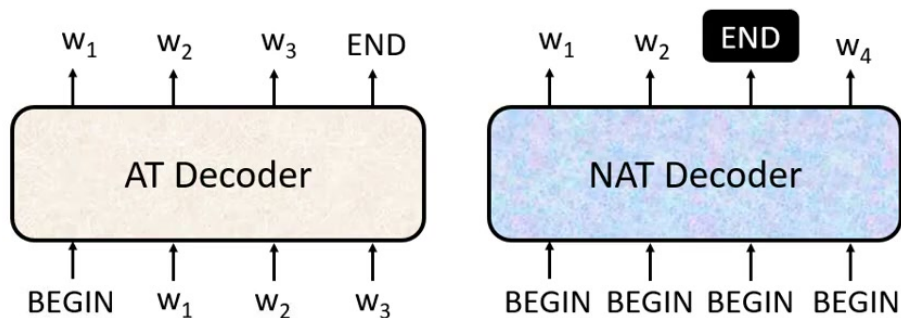
Masked self-attention 保证 decoder 不能偷看未来; END token 让 decoder 能自己决定输出长度。两者共同把 Transformer decoder 变成真正可生成序列的模块。

4 AT 与 NAT: 逐步生成还是平行生成

Autoregressive decoder 一次产生一个 token, 优点是建模自然, 缺点是推论慢, 因为第 t 步必须等第 $t - 1$ 步完成。Non-autoregressive decoder, 简称 NAT decoder, 尝试一次并行地产生多个 token。它的典型输入可能是多个 BEGIN token, 然后同时输出 $w_1, w_2, \dots$ 。

⁵ 视频画面时间区间: 约 15:25–16:05; 字幕对应 “准备一个特别的符号” “用 END 来表示” “开始跟段可以用同一个符号表示”。

AT v.s. NAT



- How to decide the output length for NAT decoder?
 - Another predictor for output length
 - Output a very long sequence, ignore tokens after END
- Advantage: parallel, controllable output length
- NAT is usually worse than AT (why? **Multi-modality**)

36

图 4: AT decoder 逐步生成并自然产生 END; NAT decoder 平行输出, 需要额外处理输出长度, 并常遇到 multi-modality 问题。⁷

4.1 NAT 的输出长度问题

NAT 不能像 AT 那样自然地生成到 END 为止, 因为它试图同时输出。常见解决方式有两种:

- 另外训练一个 length predictor, 先预测输出长度, 再让 NAT 产生对应数量 token;
- 输出一个很长的序列, 遇到 END 后忽略后面 token。

NAT 的优势是平行化, 推论速度可能更快, 输出长度也更容易控制。但它通常比 AT 难训练, 效果往往较差。

4.2 Multi-modality 直觉

NAT 的困难之一是 multi-modality。假设一句话有多个合理翻译, AT decoder 可以在第一步选择某个方向, 然后后续 token 跟着这个方向保持一致。NAT 同时预测所有位置, 每个位置可能各自选择不同模式, 导致组合起来不一致。例如有的位置像第一种翻译, 有的位置像第二种翻译, 最终句子不自然。

⁷ 视频画面时间区间: 约 22:50-23:35; 字幕对应 “NAT performance 往往不如 AT” “需要很多 trick 才能逼近 AT” “multi-modality 的问题”。

为什么 AT 容易保持一致

AT 每一步都把已经生成的前缀作为条件，因此早期选择会约束后续选择。NAT 平行生成时缺少这种逐步承诺机制，所以需要额外技巧让不同位置之间协调。

4.3 本章小结

AT 牺牲平行度换取稳定的条件生成；NAT 追求平行化，但需要额外处理长度和多模态一致性。实际系统中，NAT 往往要配合蒸馏、迭代修正或其他技巧才更接近 AT 表现。

5 Cross Attention: Encoder 与 Decoder 的桥

Decoder 不只看自己的历史输出，还必须读取 encoder 对输入序列的表示。Transformer decoder 中负责这件事的模块叫 cross attention。原始架构图里，decoder block 有三层子结构：masked multi-head self-attention、cross attention、feed-forward network。Cross attention 位于 masked self-attention 之后。

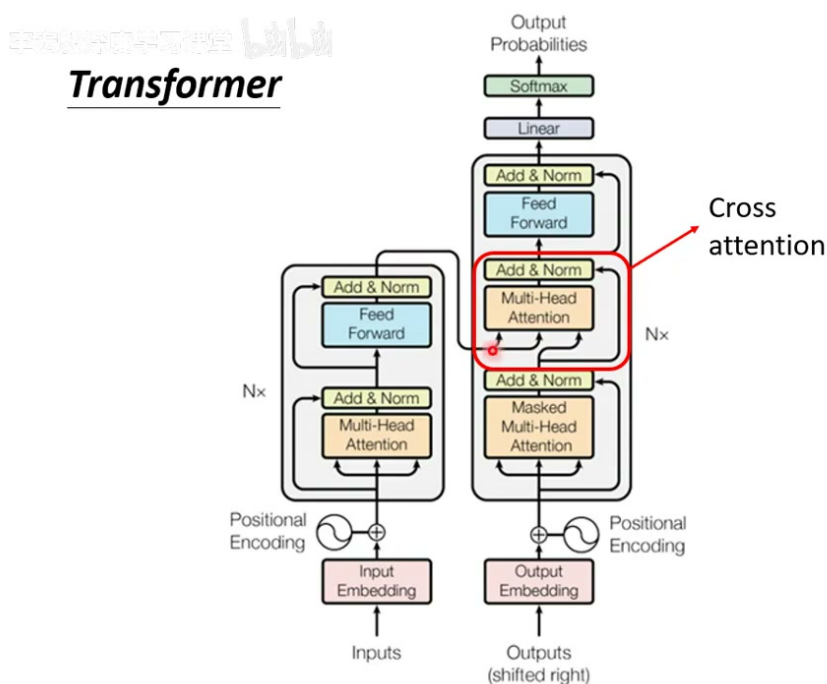


图 5: 原始 Transformer 图中，cross attention 位于 decoder block 中间，是 encoder 和 decoder 之间传递信息的桥梁。⁹

5.1 Q 来自 Decoder, K/V 来自 Encoder

Cross attention 和 self-attention 的计算形式很像，但 Q, K, V 的来源不同。在 cross attention 中：

⁹视频画面时间区间：约 24:00–24:35；字幕对应“cross attention 是连接 encoder 和 decoder 之间的桥梁”“两个输入来自 encoder，一个箭头来自 decoder”。

- query q 来自 decoder 当前状态；
- keys k_i 和 values v_i 来自 encoder 输出；
- decoder 用 query 去 encoder 输出里“查询”当前需要的信息。

具体地，若 encoder 输出为 $h_1, \dots, h_T$ ，decoder 当前状态为 s_t ，则

$$q_t = W_Q s_t, \quad k_i = W_K h_i, \quad v_i = W_V h_i.$$

注意力权重为

$$\alpha_{t,i} = \frac{\exp(q_t^\top k_i / \sqrt{d_k})}{\sum_{j=1}^T \exp(q_t^\top k_j / \sqrt{d_k})},$$

输出为

$$c_t = \sum_{i=1}^T \alpha_{t,i} v_i.$$

这个 c_t 就是 decoder 在第 t 步从输入序列中抽取出来的上下文信息。

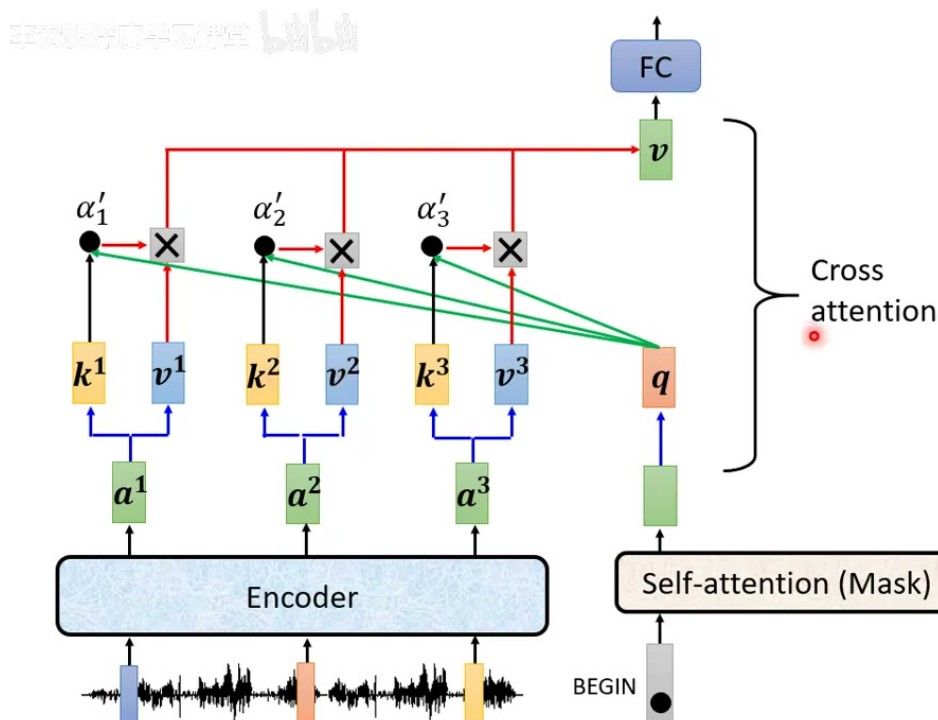


图 6: Cross attention 的 q 来自 decoder, k, v 来自 encoder; decoder 通过 query 从 encoder 表示中抽取当前步需要的信息。¹¹

5.2 Cross Attention 早于 Transformer

老师特别指出，cross attention 并不是 Transformer 才有。早期基于 LSTM 的 Seq2seq 语音辨识模型已经使用类似 attention 机制，让 decoder 生成每个输出字符时对齐输入声音片段。Transformer 的新意在于大量使用 self-attention；encoder-decoder 之间的 attention 连接则有更早历史。

¹¹ 视频画面时间区间：约 25:35–26:20；字幕对应“Q 来自 decoder，K 跟 V 来自 encoder”“decoder 凭借产生一个 Q 去 encoder 这边抽取资讯”。

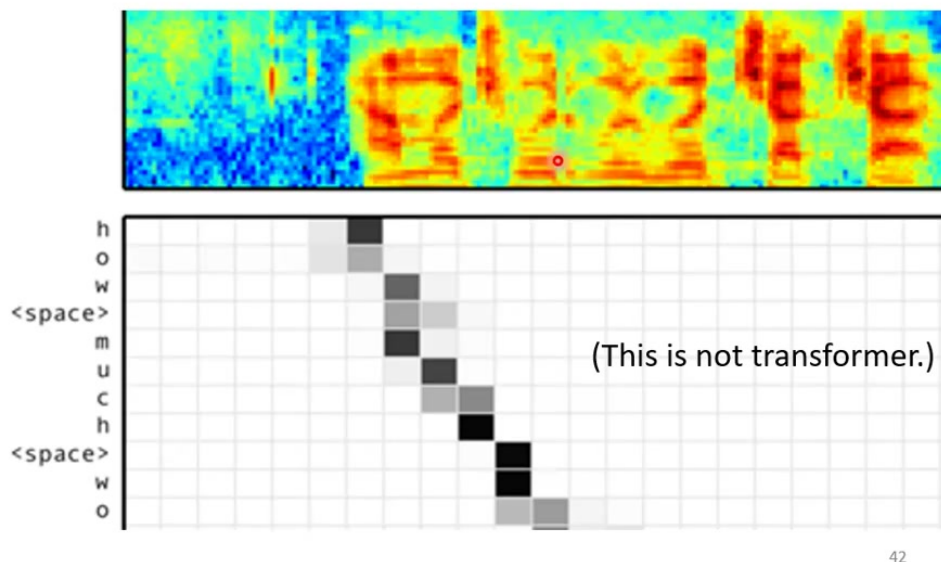


图 7: 早期 Listen, Attend and Spell 中已经有输入声音和输出字符之间的 attention 对齐; 这不是 Transformer, 但说明 cross attention 的思想更早存在。¹³

5.3 本章小结

Cross attention 让 decoder 每一步都能读取 encoder 输出。Self-attention 是同一序列内部互看; cross attention 是 decoder 用 query 去输入序列表示中取信息。Transformer decoder 正是靠 masked self-attention、cross attention 和 FFN 共同完成生成。

6 训练 Decoder: Cross Entropy 与 Teacher Forcing

训练 Seq2seq 时, 资料给出输入序列和正确输出序列。例如声音输入对应标签“机器学习”。训练目标是让 decoder 在每个位置输出的分布接近正确 token 的 one-hot vector。对第 t 个位置, 若正确 token 是 y_t^* , 损失为

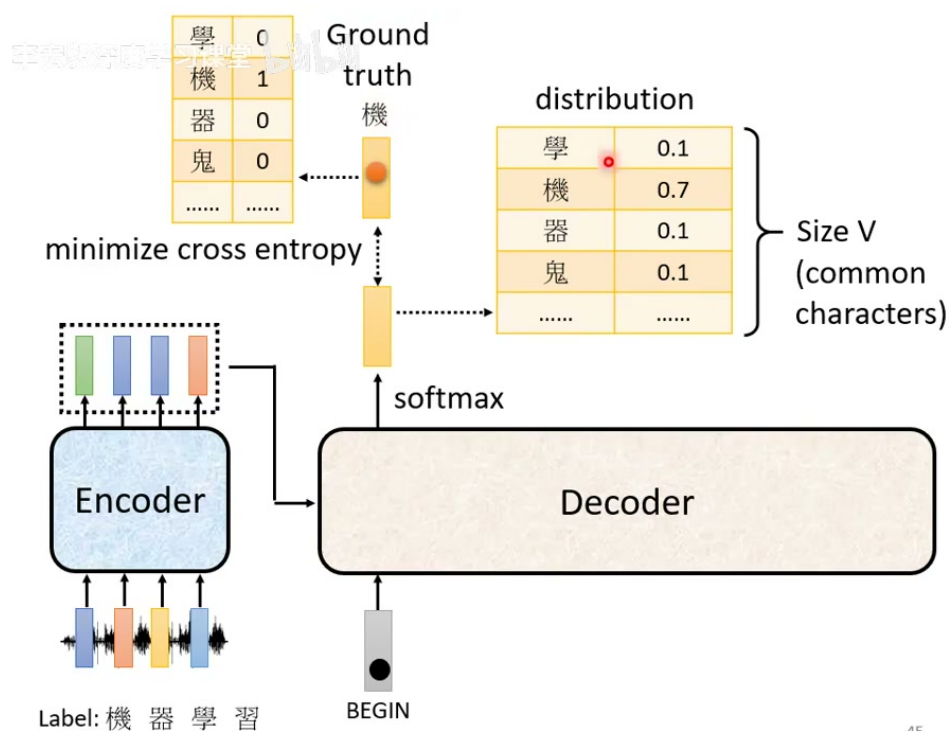
$$L_t = -\log P_\theta(y_t^* | y_{<t}^*, H).$$

整句损失为

$$L = \sum_{t=1}^{N+1} L_t,$$

其中 $N + 1$ 通常包含 END token。

¹³视频画面时间区间: 约 27:35–28:20; 字幕对应“sequence to sequence 用在语音辨识上似乎有潜力”“在有 Transformer 之前已经有 cross attention”。



45

图 8: 训练时每个输出分布都和对应的 one-hot ground truth 计算 cross entropy; 这和多分类问题本质相同。¹⁵

6.1 Teacher Forcing

训练时常用 teacher forcing: decoder 的输入前缀不是模型自己上一轮猜出的 token, 而是正确答案前缀。也就是说, 训练第 t 个输出时, 模型看到的是

$$(\text{BEGIN}, y_1^*, \dots, y_{t-1}^*),$$

而不是模型预测的

$$(\text{BEGIN}, \hat{y}_1, \dots, \hat{y}_{t-1}).$$

¹⁵ 视频画面时间区间: 约 33:25-34:00; 字幕对应 “正确答案是 one-hot vector” “希望 distribution 跟 one-hot 越接近越好” “计算 cross entropy”。

Teacher Forcing: using the ground truth as input.

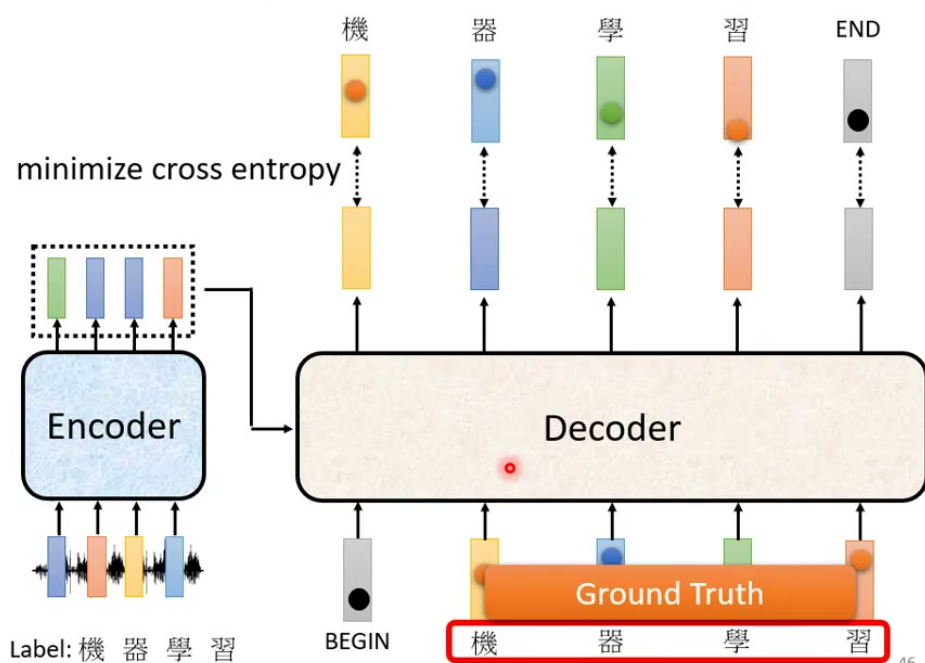


图 9: Teacher forcing 使用正确答案前缀作为 decoder 输入, 因此训练可以并行计算每个位置的 cross entropy。¹⁷

Teacher forcing 的好处是训练稳定且可平行化。因为所有正确前缀都已知, 模型可以一次处理整段目标序列, 而不用真的一步一步等待自己的输出。但它也埋下一个问题: 训练时模型总在干净前缀上学习, 测试时却必须面对自己可能产生的错误前缀。

6.2 本章小结

Decoder 训练本质上是在每个时间步做分类, 用 cross entropy 监督目标 token。Teacher forcing 让训练更容易、更快, 但也造成训练和推论输入分布不一致, 这会在后面变成 exposure bias。

7 Copy Mechanism 与 Guided Attention

课程接着介绍一些训练 Seq2seq/Transformer 的常见技巧。它们不是 Transformer 架构本身的必备组件, 而是针对特定任务的额外归纳偏置。

7.1 Copy Mechanism

在对话系统里, 如果用户说“你好, 我是库洛洛”, 机器回复“库洛洛你好, 很高兴认识你”, 最自然的行为是把用户输入中的名字复制到输出中。若用户说“小杰不能使用念能力了”, 机器也可能直接复述“不能使用念能力”这段词组, 再追问是什么意思。

¹⁷ 视频画面时间区间: 约 36:55–37:30; 字幕对应“把正确的答案当做 decoder 的输入”“训练的时候 decoder 有偷看到正确答案, 测试的时候没有”。

Copy Mechanism

Chat-bot

User: 你好，我是庫洛洛

Machine: 庫洛洛你好，很高興認識你

User: 小傑不能使用念能力了!

Machine: 你所謂的「不能使用念能力」是什麼意思？

48

图 10: Copy mechanism 让模型能把输入中的专有名词或关键短语直接复制到输出，对 chatbot 等任务很有用。¹⁹

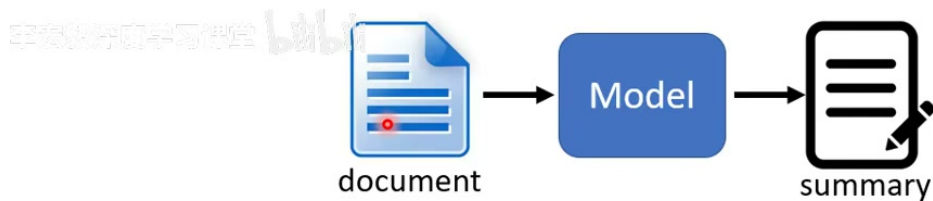
摘要任务也有同样需求。很多摘要并不是完全创造新句子，而是从原文复制重要词句，再做压缩和改写。Pointer-generator network 的直觉就是让模型在“从 vocabulary 生成”和“从 source 复制”之间切换：

$$P(w) = p_{\text{gen}} P_{\text{vocab}}(w) + (1 - p_{\text{gen}}) \sum_{i: x_i = w} \alpha_i.$$

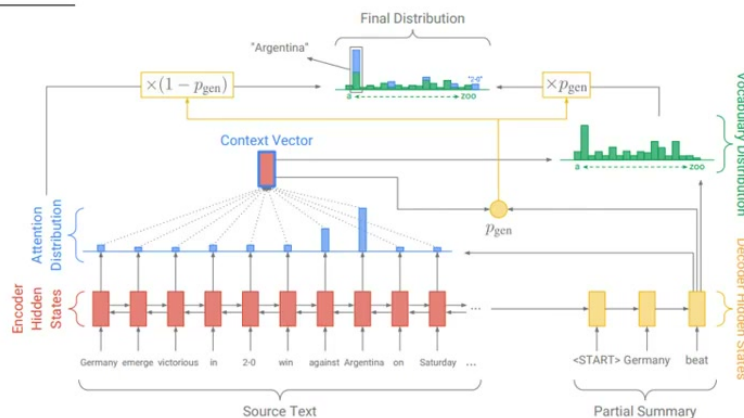
其中：

- $P_{\text{vocab}}(w)$ 是普通生成分布；
- α_i 是 source position i 的 attention weight；
- p_{gen} 决定这一刻更像“生成”还是“复制”。

¹⁹视频画面时间区间：约 39:05–39:55；字幕对应“直接把某某复制出来”“从使用者输入 copy 一些词汇当作输出”。



Summarization



<https://arxiv.org/abs/1704.04368>

图 11: 摘要任务中, 许多词汇直接来自原文; pointer-generator 类模型把 vocabulary distribution 和 attention-based copy distribution 混合。²¹

7.2 Guided Attention

某些任务中, 输入和输出天然单调对齐。例如语音识别和 TTS 通常从左到右读输入、从左到右产生输出。若 attention 在生成过程中反复跳回去或跳太远, 合成语音可能漏字、重复、卡住。Guided attention 就是把这种任务先验写进训练, 鼓励 attention 分数沿着单调方向移动。

²¹ 视频画面时间区间: 约 40:55–41:30; 字幕对应 “做摘要的时候很多词汇直接从原来的文章里面复制出来” “pointer network”。

Guided Attention

Monotonic Attention
Location-aware attention

In some tasks, input and output are monotonically aligned.
For example, speech recognition, TTS, etc.

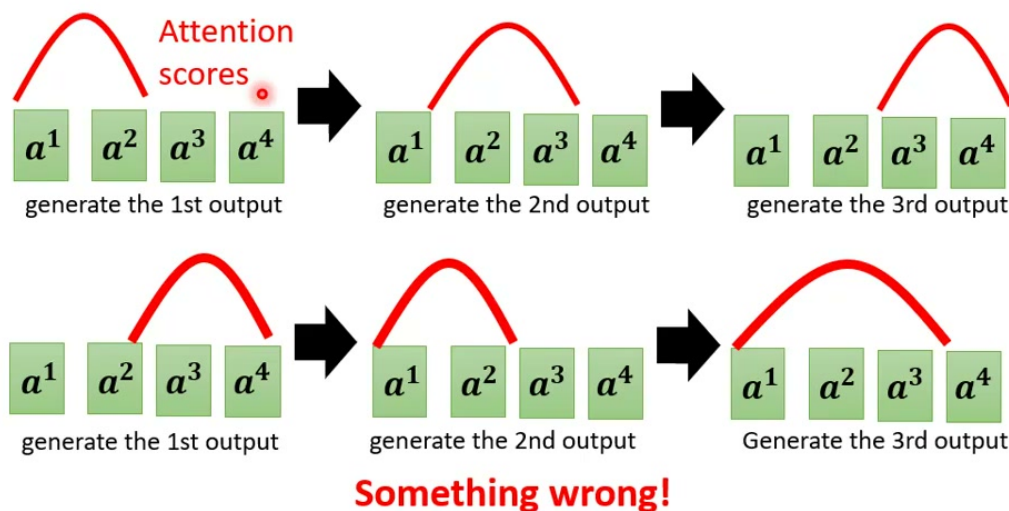


图 12: Guided attention 对 attention 形状施加约束: 在语音辨识、TTS 等单调对齐任务里, attention 应大致从左到右移动。²³

7.3 本章小结

Copy mechanism 解决“输出应直接复用输入片段”的问题; guided attention 解决“输入输出有已知对齐结构”的问题。它们都体现同一个原则: 当任务有明确结构先验时, 不必完全交给模型自己摸索。

8 解码策略: Greedy、Beam Search 与 Sampling

训练好 decoder 后, 推论时还要决定如何从每一步的分布中选 token。最简单的是 greedy decoding: 每一步都选概率最大的 token。但局部最优不等于全局最优。第一步选概率最大的 token, 可能导致后续路径整体概率较低。

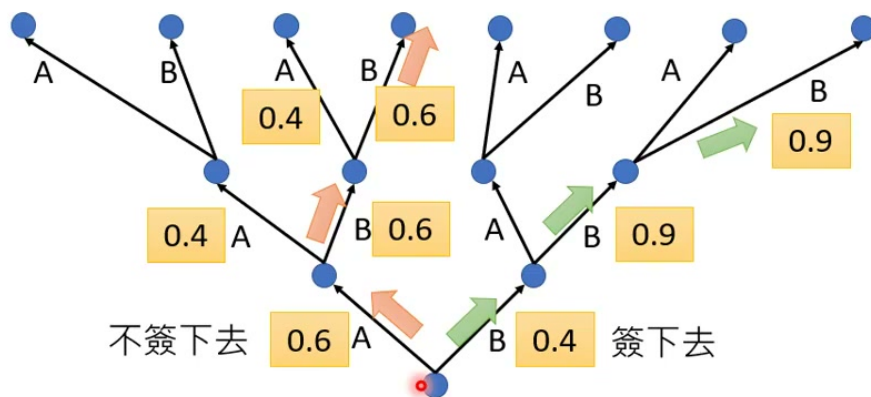
Beam search 保留多个候选前缀, 而不是只保留一个。若 beam size 为 B , 每一步扩展所有候选, 再保留分数最高的 B 个。常见序列分数是 log probability 之和:

$$\text{score}(y_{1:t}) = \sum_{i=1}^t \log P(y_i | y_{<i}, H).$$

²³ 视频画面时间区间: 约 46:35–47:20; 字幕对应“attention 有问题会合不出好的结果”“要求机器学到 attention 应该由左向右”。

Beam Search

Assume there are only two tokens ($V=2$).



The **red** path is **Greedy Decoding**.

The **green** path is the best one.

Not possible to check all the paths ... → Beam Search

图 13: Greedy decoding 只走红色局部最优路径; beam search 用近似搜索保留多条候选, 尝试接近绿色全局较优路径。²⁵

8.1 什么时候 Beam Search 有用

Beam search 对答案明确的任务更有帮助, 例如语音辨识、机器翻译等。目标通常只有少数合理答案, 找到高概率序列往往能提升结果。对开放式生成、故事续写、聊天等任务, 过强的 beam search 可能产生无聊、重复、模板化输出, 因为“最可能的句子”未必是最有趣或最自然的句子。

8.2 Sampling

当任务需要多样性时, 可以从分布中随机采样, 而不是总取最大值。Sampling 可以让生成结果更丰富, 但也会引入不稳定。实际系统常配合 temperature、top-k、top-p 等方法控制随机性。

²⁵ 视频画面时间区间: 约 50:35–51:15; 字幕对应“无法穷举所有路径”“每个地方分叉都是很多可能”“beam search 找 approximate solution”。

The Curious Case of Neural Text Degeneration

<https://arxiv.org/abs/1904.09751>

Beam Search, $b=32$:

Pure Sampling:

They were cattle called **Bolivian Cavaliers**; they live in a remote desert uninterrupted by town, and they speak **huge, beautiful, paradisiacal Bolivian linguistic** thing. They say, 'Lunch, marge.' They don't tell what the lunch is," director Professor Chuperas Ormwell told Sky News. "They've only been talking to scientists, like we're being interviewed by TV reporters. We don't even stick around to be interviewed by TV reporters. Maybe that's how they figured out that they're cosplaying as the Bolivian Cavaliers."

Randomness is needed for decoder when generating sequence in some tasks (e.g., sentence completion, TTS).

图 14: 开放式文本生成中, beam search 可能过于保守; 适度随机性有助于产生更自然、多样的输出。²⁷

8.3 评价指标与强化学习

老师还提到：如果你真正关心的是 BLEU 等句子级评价指标，逐 token 的 cross entropy 未必直接优化这些指标。这时可以考虑 reinforcement learning，把不可微或句子级指标当作 reward 来优化。不过这条路通常更难训练，也需要谨慎设计 reward，否则模型可能学会钻指标漏洞。

Cross entropy 优化的是下一个 token 的条件概率；BLEU、ROUGE、人类偏好等指标评价的是整段输出。两者相关但不相同。许多生成模型问题都来自这个错位：局部概率高，不代表整体结果好。

8.4 本章小结

Greedy 简单但短视；beam search 是近似全局搜索，适合答案较明确的任务；sampling 增加多样性，适合开放式生成。解码策略不是训练后的细枝末节，而会显著影响最终表现。

²⁷ 视频画面时间区间：约 53:10-53:40；字幕对应“需要机器发挥一点创造力”“不是只有一个答案的任务”。

9 Exposure Bias 与 Scheduled Sampling

Teacher forcing 带来训练和推论不一致。训练时 decoder 总是看到正确前缀；推论时 decoder 看到自己的预测前缀。若模型从没见过训练时见过错误前缀，测试时一旦早期出错，就可能一步错、步步错。这个现象叫 exposure bias。

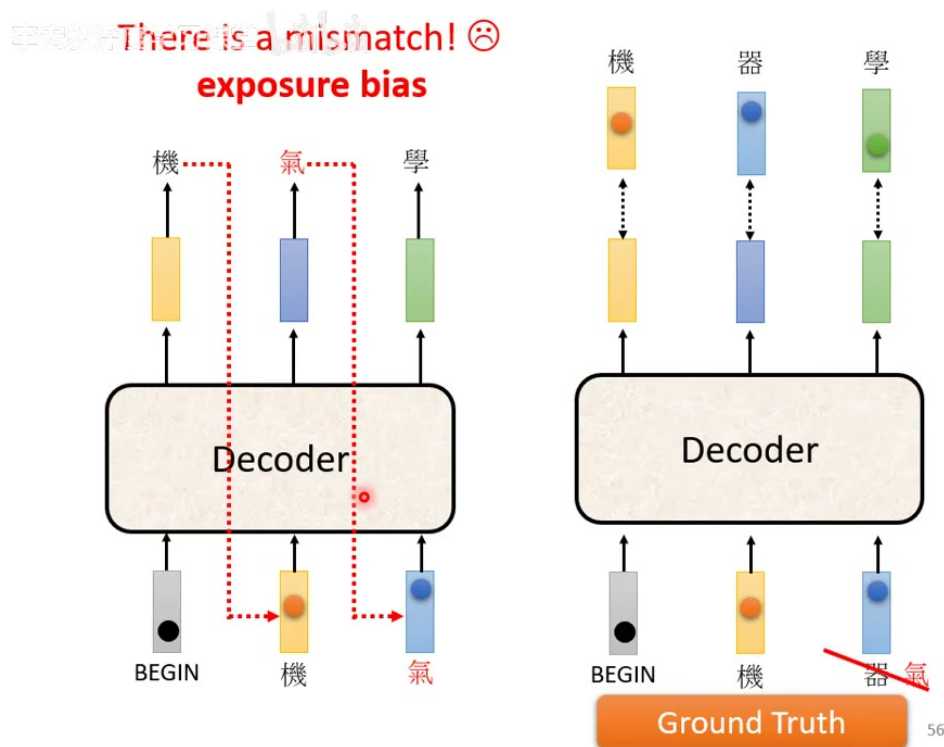


图 15: Exposure bias: 训练时 decoder 输入是正确前缀，测试时输入却来自模型自己的预测；一旦预测错，后续状态就偏离训练分布。²⁹

9.1 Scheduled Sampling 的直觉

一种直觉解法是：训练时不要永远喂正确 token，偶尔把模型自己预测的 token 喂回去。这样模型会在训练中见到一些错误前缀，学会从不完美状态恢复。这个方法叫 scheduled sampling。

²⁹ 视频画面时间区间：约 58:50–59:20；字幕对应“训练的时候都是正确的，测试的时候模型看到自己的输出”“这个不一致叫 exposure bias”。

Scheduled Sampling

- Original Scheduled Sampling

<https://arxiv.org/abs/1506.03099>

- Scheduled Sampling for Transformer

<https://arxiv.org/abs/1906.07651>

- Parallel Scheduled Sampling

<https://arxiv.org/abs/1906.04331>

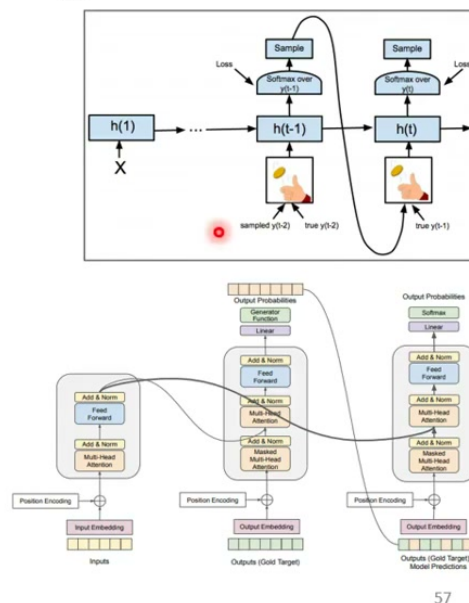


图 16: Scheduled sampling 让训练时的 decoder 偶尔接收模型采样输出，试图缓解 teacher forcing 和推论之间的 mismatch。³¹

不过传统 scheduled sampling 会破坏 Transformer 的平行训练能力。因为一旦某个位置输入依赖模型上一位置的实际采样结果，训练就不能简单地一次并行算完所有位置。因此，Transformer 里的 scheduled sampling 需要专门设计，不能直接照搬早期 RNN/LSTM 的做法。

Scheduled Sampling 不是学习率排程

课程中特别提醒：scheduled sampling 和 learning rate schedule 是两件完全不同的事。前者改变 decoder 训练时看到的输入 token；后者改变优化器更新参数的步长。

9.2 本章小结

Exposure bias 来自 teacher forcing：训练看正确前缀，推论看模型前缀。Scheduled sampling 的思想是让训练更像推论，但在 Transformer 中会牵涉平行化和训练稳定性，需要额外设计。

³¹ 视频画面时间区间：约 59:35–60:15；字幕对应“偶尔给它一些错的东西”“这一招叫 scheduled sampling”“对 Transformer 来说另有招数”。

10 总结与延伸

本讲把 Transformer 的 decoder 和训练技巧补完整。若把两讲合起来看，Transformer 的完整结构可以概括为：

$$\text{Encoder} : \text{Embed} + \text{Pos} \rightarrow (\text{SelfAttn} + \text{FFN})^N,$$

$$\text{Decoder} : \text{ShiftedEmbed} + \text{Pos} \rightarrow (\text{MaskedSA} + \text{CrossAttn} + \text{FFN})^N.$$

Decoder 这半边最重要的概念有五个：

- Autoregressive generation：一步一步产生 token；
- Masked self-attention：不能偷看未来 token；
- Cross attention：用 decoder query 读取 encoder key/value；
- Teacher forcing：训练时用正确前缀，换取稳定和并行；
- Exposure bias：训练/推论前缀分布不一致，可能造成错误传播。

本讲最重要的理解

Transformer decoder 是一个条件生成器。它既要遵守因果顺序，又要读取输入序列，还要在训练时被逐 token 监督。很多“技巧”不是装饰，而是在补 decoder 的真实缺口：复制输入、保持单调对齐、近似搜索、增加随机性、缓解 exposure bias。

从工程角度看，Transformer 不是一张架构图就结束了。训练目标、teacher forcing、解码策略、停止条件和任务先验都会强烈影响最终系统表现。学会 encoder/decoder 的方块连接只是第一步；真正能用好 Seq2seq 模型，还要理解生成过程中的这些细节。

10.1 拓展阅读

- Attention Is All You Need：原始 Transformer；
- Listen, Attend and Spell：早期语音辨识中的 encoder-decoder attention；
- Pointer-generator Network：摘要中的 copy mechanism；
- The Curious Case of Neural Text Degeneration：beam search 与 sampling 的生成质量问题；
- Scheduled Sampling：缓解 exposure bias 的早期方法。